WDJ - QB UNIT 5 (5- marks)

1. Define ORM and explain its advantages.

1. ORM stands for Object Relational Mapping and it is used to map Java objects to database tables. It acts as a bridge between object-oriented programming and relational databases.
2. In Java applications, data is represented using objects and classes. In databases, data is stored in rows and tables.
3. Without ORM, developers need to write SQL queries manually for database operations. With ORM, developers can perform operations using Java objects and methods.
4. ORM reduces the amount of boilerplate SQL code written by developers. This helps in improving productivity and saves development time.
5. ORM simplifies CRUD operations like insert, update, delete, and retrieve. Developers can manage database data easily using Java code.
6. ORM makes applications more maintainable and easier to understand. It also helps applications become database-independent.
7. ORM frameworks automatically handle the conversion between Java objects and database records. This reduces coding complexity and improves software quality.

2. What is JPA?

1. JPA stands for Java Persistence API and it is a Java specification used for managing relational data. It provides rules and standards for ORM in Java applications.
2. JPA helps in mapping Java classes to database tables using annotations. It also provides APIs for performing CRUD operations.
3. JPA is only a specification and not a complete implementation. It defines standards that ORM frameworks must follow.
4. Developers use JPA annotations like @Entity, @Table, @Id, and @Column for mapping entities. These annotations simplify database interaction in Java applications.
5. JPA helps developers avoid writing large amounts of SQL queries manually. This makes development faster and easier.
6. To use JPA in applications, an implementation framework is required. Popular JPA implementations include Hibernate, EclipseLink, and OpenJPA.
7. In Spring Boot applications, JPA is commonly used with Hibernate for database operations. The flow is Java Application → JPA → Hibernate → Database.

3. Explain Hibernate as a JPA implementation.

1. Hibernate is an ORM framework used in Java applications for database operations. It maps Java classes and objects to database tables and records.
2. JPA is only a specification that defines rules for ORM. Hibernate provides the actual implementation of those JPA rules and interfaces.
3. Hibernate works between the Java application and the database layer. It converts Java-based operations into SQL queries automatically.
4. Developers use JPA annotations in entity classes and Hibernate executes the actual SQL operations. This reduces the need for writing SQL manually.
5. Hibernate automatically generates SQL queries for insert, update, delete, and select operations. This improves productivity and simplifies development.
6. Hibernate supports features like caching, relationship mapping, and database independence. It can work with databases such as MySQL, PostgreSQL, and Oracle.
7. In Spring Boot, Hibernate is the default JPA provider when spring-boot-starter-data-jpa dependency is added. Spring Boot automatically configures JPA, Hibernate, and the data source.

4. What is an Entity in JPA?

1. An Entity in JPA is a Java class that represents a database table. Each object of the entity class represents one record in the table.
2. The @Entity annotation is used to mark a Java class as an entity. This tells JPA that the class should be mapped to a database table.
3. The @Table annotation is used to specify the table name in the database. It helps in connecting the entity class with a particular table.
4. Every entity must contain a primary key field. The @Id annotation is used to define the primary key in the entity class.
5. The @Column annotation is used to map class fields to table columns. This makes it easy to store and retrieve data from the database.
6. Entity classes are important in ORM because they connect Java objects with relational database tables. They simplify database interaction using Java code instead of SQL.
7. In Spring Boot applications, entities work together with repositories and Hibernate for CRUD operations. This helps developers build database-driven applications easily.

5. Explain entity mapping using annotations.

1. Entity mapping is the process of connecting Java classes with database tables using annotations. It helps ORM frameworks understand how objects should be stored in the database.
2. In JPA, annotations are used to define mappings between class fields and table columns. These annotations are provided by the Java Persistence API.
3. The @Entity annotation is used to mark a Java class as an entity. It tells JPA that the class represents a database table.
4. The @Table annotation is used to specify the table name in the database. This annotation helps map the entity class to a particular table.
5. The @Id annotation is used to define the primary key of the entity. Every entity class must contain a primary key field.
6. The @Column annotation maps a class field to a specific table column. It allows developers to customize column names in the database.
7. Entity mapping using annotations simplifies database operations and reduces manual SQL coding. It also improves maintainability and readability of the application.

6. What is Spring Data JPA?

1. Spring Data JPA is a framework used in Spring Boot for performing database operations easily. It simplifies data access using JPA and repository interfaces.
2. In traditional applications, developers needed to write SQL queries manually for database operations. Spring Data JPA reduces this work by providing built-in methods.
3. Spring Data JPA provides repository interfaces such as JpaRepository for interacting with the database. These interfaces contain ready-made CRUD methods.
4. Developers can perform operations like insert, update, delete, and retrieve without writing SQL queries manually. This makes development faster and simpler.
5. Spring Data JPA works together with JPA and Hibernate in Spring Boot applications. Hibernate handles the actual SQL generation internally.
6. To create a repository, developers extend the JpaRepository interface with the entity class and primary key type. Spring automatically provides the required implementations.
7. Spring Data JPA improves productivity, reduces boilerplate code, and makes applications easier to maintain. It is widely used in modern Spring Boot applications.

7. Explain CRUD operations using Spring Data JPA.

1. CRUD operations refer to Create, Read, Update, and Delete database operations. Spring Data JPA provides built-in methods to perform these operations easily.
2. The save() method is used to insert new data into the database. It can also update existing records when the object already exists.
3. The findAll() method retrieves all records from the database table. It returns the data as a list of objects.
4. The findById() method is used to retrieve a single record using its primary key. It helps developers fetch specific data quickly.
5. The deleteById() method deletes a record from the database using its ID. This method simplifies delete operations without writing SQL.
6. CRUD operations in Spring Data JPA are performed through repository interfaces. Developers only need to call the required methods in Java code.
7. Spring Data JPA automatically generates SQL queries for these operations using Hibernate. This reduces coding effort and improves development speed.

8. What are repository interfaces in Spring Data JPA?

1. A repository interface acts as the data access layer between the application and the database. It helps applications interact with the database easily.
2. In Spring Data JPA, repository interfaces are created by extending interfaces like JpaRepository. These interfaces provide built-in database operation methods.
3. Developers do not need to write SQL queries manually when using repository interfaces. Spring Data JPA automatically handles database interactions.
4. The JpaRepository interface provides methods such as save(), findAll(), findById(), and deleteById(). These methods are used for CRUD operations.
5. While creating a repository interface, developers specify the entity class and the primary key type. This helps Spring identify which table the repository will manage.
6. Repository interfaces reduce boilerplate code and simplify database programming. They also improve code readability and maintainability.
7. In Spring Boot applications, repository interfaces work together with JPA and Hibernate for automatic SQL generation and database management.

9. Why is transaction management required?

1. Transaction management is required to maintain data consistency in database operations. It ensures that all operations in a transaction are completed successfully together.
2. A transaction is treated as a single unit of work in the database. If one operation fails, all other operations are rolled back automatically.
3. Transaction management prevents incomplete or incorrect data from being stored in the database. This helps maintain the reliability of the application.
4. It is especially important in applications involving multiple related database operations. Examples include banking systems, order processing, and payment systems.
5. In a money transfer process, one account is debited and another is credited. If the second operation fails, the first operation must also be cancelled.
6. Without transaction management, database inconsistency may occur during system failures or exceptions. This can create incorrect or corrupted records.
7. Spring Framework provides transaction management support using the @Transactional annotation. This simplifies handling transactions in Spring Boot applications.

10. What is @Transactional annotation?

1. @Transactional is an annotation used in Spring Framework for transaction management. It ensures that a method executes within a database transaction.
2. When a method is marked with @Transactional, Spring automatically starts a transaction before executing the method. After execution, the transaction is either committed or rolled back.
3. If all database operations execute successfully, the transaction is committed permanently. This means the changes are saved in the database.
4. If any exception occurs during execution, Spring rolls back the transaction automatically. This prevents partial or inconsistent data storage.
5. The @Transactional annotation is mainly used in service methods that contain multiple database operations. It helps maintain data consistency and reliability.
6. In a money transfer example, withdrawal and deposit operations are placed in one transaction. If deposit fails, the withdrawal operation is also cancelled automatically.
7. Using @Transactional reduces manual transaction handling code in applications. It makes transaction management simple and efficient in Spring Boot.

11. Define exception handling in RESTful services.

1. Exception handling in RESTful services is the process of managing errors that occur during API execution. It helps applications return meaningful responses instead of crashing.
2. Errors in REST APIs may occur because of invalid input, missing resources, database problems, or server issues. Proper exception handling improves API usability and reliability.
3. Instead of displaying technical errors like NullPointerException, APIs return proper HTTP status codes and messages. This helps clients understand the problem clearly.
4. Common HTTP error codes used in REST APIs include 400 Bad Request, 404 Not Found, and 500 Internal Server Error. These codes indicate different types of errors.
5. Exception handling allows developers to control how errors are presented to the user. It also improves debugging and application maintenance.
6. In Spring Framework, exception handling can be done using annotations like @ExceptionHandler and @ControllerAdvice. These features simplify error management in REST APIs.
7. Proper exception handling improves the overall user experience and software quality. It ensures that APIs respond in a structured and professional manner.

12. What is @ControllerAdvice?

1. @ControllerAdvice is an annotation in Spring Framework used for global exception handling. It allows developers to manage exceptions for all controllers from one central class.
2. Without @ControllerAdvice, exception handling code needs to be written separately in every controller. This increases code duplication and complexity.
3. Using @ControllerAdvice improves code reusability and application maintainability. It helps keep controller classes clean and organized.
4. Inside a class marked with @ControllerAdvice, developers use @ExceptionHandler methods to handle specific exceptions. These methods return meaningful error responses.
5. Global exception handling ensures that all API errors are handled consistently across the application. This improves the reliability of RESTful services.
6. @ControllerAdvice can also work with @ResponseStatus to return proper HTTP status codes. For example, it can return 404 Not Found when a resource is missing.
7. In Spring Boot applications, @ControllerAdvice is widely used for centralized error management. It simplifies exception handling and improves API response quality.

13. What are custom exceptions?

1. Custom exceptions are user-defined exceptions created for specific application errors. They help developers handle special situations in a better and more meaningful way.
2. In Spring Boot applications, custom exceptions are commonly used for cases like student not found or invalid account. These exceptions make error handling more organized.
3. Custom exceptions are usually created by extending the RuntimeException class. This allows developers to define their own exception classes easily.
4. A custom exception can include a message explaining the error clearly. This message is returned to the client when the exception occurs.
5. Developers throw custom exceptions using the throw keyword in Java. This helps stop execution when a specific problem is detected.
6. In REST APIs, custom exceptions are often handled using @ExceptionHandler and @ResponseStatus. These annotations return proper HTTP status codes and messages.
7. Custom exceptions improve application readability, maintainability, and debugging. They also help provide user-friendly error responses in APIs.

14. Explain HTTP status codes in REST APIs.

1. HTTP status codes are standard response codes returned by REST APIs to indicate the result of a request. They help clients understand whether the request was successful or failed.
2. Status codes improve communication between the client and the server. They provide clear information about the response condition.
3. The 200 OK status code indicates that the request was completed successfully. It is commonly returned for successful API operations.
4. The 400 Bad Request status code is returned when the client sends invalid input or incorrect data. It tells the client that the request format is wrong.
5. The 404 Not Found status code indicates that the requested resource does not exist. For example, it is returned when a student record is missing.
6. The 500 Internal Server Error status code shows that a problem occurred on the server side. This usually happens because of unexpected exceptions or server failures.
7. In Spring Boot, HTTP status codes can be managed using annotations like @ResponseStatus. Proper status codes improve API usability and error handling.

15. What is testing in Spring Boot?

1. Testing in Spring Boot is the process of checking whether the application works correctly as expected. It helps developers identify and fix errors during development.
2. Spring Boot testing is used to verify application components and API responses. It ensures that all features work properly before deployment.
3. Spring Boot provides built-in support for testing using JUnit and Spring testing tools. These tools simplify the testing process for developers.
4. Testing helps detect bugs early in the development cycle. Early bug detection reduces development and maintenance costs.
5. Developers can test methods, controllers, services, and REST APIs in Spring Boot applications. This improves the reliability of the software.
6. Spring Boot supports automated testing for checking application behavior repeatedly. Automated tests save time and reduce manual testing effort.
7. Proper testing improves software quality and increases confidence in the application. It ensures that the application behaves correctly after code changes.

16. What is JUnit?

1. JUnit is a testing framework used in Java for unit testing applications. It helps developers test small units of code such as methods.
2. Unit testing checks whether individual methods work correctly. Each test method focuses on testing one specific functionality.
3. In JUnit, methods are marked with the @Test annotation to identify them as test cases. These test methods are executed automatically during testing.
4. JUnit provides assertion methods like assertEquals() for checking expected and actual outputs. This helps verify whether the method works properly.
5. JUnit testing helps developers find bugs quickly during development. It improves application reliability and software quality.
6. JUnit is widely used in Spring Boot applications for testing services, controllers, and utility methods. It supports automated testing efficiently.
7. Using JUnit reduces manual testing effort and supports continuous development practices. It also makes applications easier to maintain and debug.

17. Explain JUnit lifecycle.

1. JUnit lifecycle refers to the sequence of execution of test methods and lifecycle annotations. It controls what happens before and after test execution.
2. JUnit provides annotations such as @BeforeAll, @BeforeEach, @AfterEach, and @AfterAll. These annotations help manage test setup and cleanup activities.
3. @BeforeAll runs once before all test methods execute. It is commonly used for initializing shared resources.
4. @BeforeEach runs before every test method. It helps prepare the environment required for each test.
5. @AfterEach runs after every test method execution. It is used to clean temporary data or release resources.
6. @AfterAll runs once after all test methods are completed. It is mainly used for final cleanup operations.
7. The lifecycle order in JUnit is @BeforeAll, @BeforeEach, @Test, @AfterEach, and @AfterAll. This structure helps maintain organized and reliable test execution.

18. Why is automated testing important?

1. Automated testing means running test cases automatically to verify application behavior. It reduces the need for repeated manual testing.
2. Automated testing saves time because tests can run quickly and repeatedly. This is useful when applications are updated frequently.
3. It improves reliability by reducing human errors during testing. Automated tests produce more consistent results compared to manual testing.
4. Automated testing helps detect bugs early during development. Early bug detection improves software quality and reduces maintenance costs.
5. After modifying application code, automated tests verify whether existing functionality still works correctly. This prevents accidental errors in the system.
6. Automated testing increases developer confidence while making code changes. Developers can update applications without fear of breaking existing features.
7. In Spring Boot applications, automated testing improves overall software quality and stability. It ensures that APIs and application components work correctly over time.

(10 – marks)

1. Explain ORM concept and JPA architecture with advantages.

1. ORM stands for Object Relational Mapping and it is used to connect Java objects with database tables. It acts as a bridge between object-oriented programming and relational databases.
2. In Java applications, data is stored in objects and classes while databases store data in tables and rows. ORM helps convert Java objects into database records automatically.
3. Without ORM, developers need to write SQL queries manually for insert, update, delete, and retrieve operations. With ORM, developers can perform database operations using Java objects and methods.
4. JPA stands for Java Persistence API and it is a specification used for managing relational data in Java applications. It defines rules and standards for ORM and database interaction.
5. JPA provides annotations such as @Entity, @Table, @Id, and @Column for mapping Java classes to database tables. These annotations simplify entity mapping and database management.
6. JPA itself is not an implementation and requires frameworks like Hibernate, EclipseLink, or OpenJPA. Hibernate is the most commonly used JPA implementation in Spring Boot applications.
7. The architecture flow of JPA is Java Application → JPA → Hibernate → Database. The application uses JPA standards while Hibernate performs the actual SQL operations.
8. ORM and JPA reduce boilerplate SQL code and improve developer productivity. They simplify CRUD operations and make applications easier to maintain.
9. ORM provides database independence because applications can work with different databases with minimal changes. This improves portability and flexibility of applications.
10. JPA and ORM improve software maintainability by reducing coding complexity and increasing readability. They also help developers build scalable and efficient database-driven applications.

2. Explain Hibernate framework and its role as JPA implementation.

1. Hibernate is an ORM framework used in Java applications for handling database operations. It maps Java classes and objects to database tables and records.
2. Hibernate reduces the need for writing SQL queries manually in applications. Developers can work with Java objects while Hibernate handles SQL generation internally.
3. JPA is only a specification that defines standards and rules for ORM. Hibernate provides the actual implementation of JPA interfaces and annotations.
4. Hibernate works between the Java application and the database system. It converts JPA-based operations into SQL queries automatically.
5. In Hibernate architecture, the flow is Java Application → JPA → Hibernate → Database. This structure simplifies communication between applications and databases.
6. Hibernate supports automatic table mapping using annotations such as @Entity, @Table, and @Id. These annotations help connect Java classes with database tables.
7. Hibernate automatically generates SQL queries for insert, update, delete, and select operations. This reduces coding effort and increases development speed.
8. Hibernate provides features like caching support and relationship mapping between entities. It supports relationships such as @OneToOne, @OneToMany, and @ManyToMany.
9. Hibernate is database-independent and works with databases like MySQL, PostgreSQL, and Oracle. Changing the database requires only minimal code modifications.
10. In Spring Boot, Hibernate is the default JPA provider when spring-boot-starter-data-jpa dependency is added. Spring Boot automatically configures Hibernate, JPA, and the data source.
11. Hibernate improves application maintainability and productivity by reducing JDBC code. It also supports automatic schema generation and easier database integration.
12. Because of its simplicity, efficiency, and strong ORM support, Hibernate is one of the most popular JPA implementation frameworks used in enterprise applications.

3. Explain entity mapping in JPA using annotations with examples.

1. Entity mapping in JPA is used to connect Java classes with database tables. It helps store Java objects as records in relational databases.
2. JPA uses annotations to define mappings between classes and tables. These annotations simplify database operations in Java applications.
3. The @Entity annotation marks a Java class as an entity. It tells JPA that the class represents a database table.
4. The @Table annotation is used to specify the table name in the database. It maps the entity class to a particular table.
5. The @Id annotation defines the primary key field of the entity. Every entity must contain a unique primary key.
6. The @Column annotation maps class fields to database columns. It also allows customization of column names.
7. Example:

```
import jakarta.persistence.*;

@Entity
@Table(name="student")
public class Student {

  @Id
  private int id;

  @Column(name="student_name")
  private String name;
}
```

8. In this example, the Student class is mapped to the student table. The id field becomes the primary key and name maps to the student_name column.

4. Describe CRUD operations using Spring Data JPA with example.

1. CRUD stands for Create, Read, Update, and Delete operations in database applications. Spring Data JPA provides built-in methods for performing these operations.
2. Developers can perform CRUD operations without writing SQL queries manually. Spring Data JPA works with Hibernate to generate SQL automatically.
3. The save() method is used to insert or update records in the database. It simplifies create and update operations.
4. The findAll() method retrieves all records from the table. The findById() method retrieves a specific record using its ID.
5. The deleteById() method deletes a record using the primary key value. These methods are available through JpaRepository.
6. Example repository:

```
public interface StudentRepository
extends JpaRepository<Student, Integer> {
}
```

7. Example CRUD operations:

```
// CREATE
repo.save(student);

// READ
repo.findAll();
repo.findById(101);

// UPDATE
repo.save(student);

// DELETE
repo.deleteById(101);
```

8. Spring Data JPA reduces coding complexity and improves productivity. It also makes database operations easier and faster in Spring Boot applications.

5. Explain repository interfaces and their role in Spring Data JPA.

1. A repository interface acts as the data access layer between the application and the database. It helps applications perform database operations easily.
2. In Spring Data JPA, repository interfaces are created by extending interfaces like JpaRepository. These interfaces provide built-in CRUD methods automatically.
3. Developers do not need to write SQL queries manually while using repository interfaces. Spring Data JPA internally handles database interaction.
4. Repository interfaces provide methods such as save(), findAll(), findById(), and deleteById(). These methods simplify insert, retrieve, update, and delete operations.
5. Example repository interface:

```
public interface StudentRepository
extends JpaRepository<Student, Integer> {
}
```

6. In this example, Student is the entity class and Integer is the primary key type. Spring automatically creates the implementation for this interface.
7. Repository interfaces reduce boilerplate code and improve maintainability of applications. They also increase development speed in Spring Boot projects.
8. Repository interfaces work together with JPA and Hibernate for automatic SQL generation. This makes database programming easier and more efficient.

6. Explain transaction management in Spring Boot using @Transactional.

1. Transaction management is used to ensure that database operations are completed safely and consistently. It treats multiple operations as a single unit of work.
2. In Spring Boot, transaction management is handled using the @Transactional annotation. This annotation ensures that methods execute inside a transaction.
3. When a method marked with @Transactional starts, Spring automatically begins a transaction. After execution, the transaction is either committed or rolled back.
4. If all operations execute successfully, the transaction is committed permanently to the database. If an exception occurs, all changes are rolled back automatically.
5. Transaction management helps prevent inconsistent data in the database. It is very important in banking, payment, and order-processing systems.
6. Example:

```
@Transactional
public void transferMoney() {
   accountRepository.withdraw();
   accountRepository.deposit();
}
```

7. In this example, both withdraw and deposit operations are part of one transaction. If the deposit operation fails, the withdrawal is also cancelled automatically.
8. Using @Transactional reduces manual transaction handling code and improves reliability. It also helps maintain data consistency in Spring Boot applications.

7. Explain exception handling in RESTful services with @ControllerAdvice and @ExceptionHandler.

1. Exception handling in RESTful services is used to manage errors that occur during API execution. It helps APIs return meaningful responses instead of crashing.
2. Errors in REST APIs may happen because of invalid input, missing resources, database issues, or server problems. Proper exception handling improves API usability and reliability.
3. @ExceptionHandler is used to handle specific exceptions in Spring Boot applications. It allows developers to define custom responses for errors.
4. @ControllerAdvice is used for global exception handling across all controllers. It removes the need to write exception handling code in every controller.
5. Example custom exception:

```
public class StudentNotFoundException
extends RuntimeException {

  public StudentNotFoundException(String message) {
    super(message);
  }
}
```

6. Example global exception handler:

```
@ControllerAdvice
public class GlobalExceptionHandler {

  @ExceptionHandler(StudentNotFoundException.class)
  @ResponseStatus(HttpStatus.NOT_FOUND)
  public String handleStudentNotFound(
      StudentNotFoundException ex) {
    return ex.getMessage();
  }
}
```

7. In this example, if a student record is not found, the API returns a proper error message with HTTP status 404 Not Found. This improves the quality and readability of API responses.
8. Using @ControllerAdvice and @ExceptionHandler improves maintainability and centralized error management. It also helps provide consistent responses across the application.

9. Design a REST API with proper exception handling and HTTP status codes.

1. A REST API is used to allow communication between clients and servers using HTTP methods. Proper exception handling makes the API reliable and user-friendly.
2. HTTP status codes are used to indicate the result of API requests. Common codes include 200 OK, 400 Bad Request, 404 Not Found, and 500 Internal Server Error.
3. Example REST controller:

```
@RestController
@RequestMapping("/students")
public class StudentController {

  @GetMapping("/{id}")
  public Student getStudent(@PathVariable int id) {

    Student s = repo.findById(id).orElse(null);

    if(s == null) {
      throw new StudentNotFoundException(
        "Student not found");
    }

    return s;
  }
}
```

4. In this API, a student is searched using the student ID. If the student does not exist, a custom exception is thrown.
5. Example exception handler:

```
@ControllerAdvice
public class GlobalExceptionHandler {

  @ExceptionHandler(StudentNotFoundException.class)
  @ResponseStatus(HttpStatus.NOT_FOUND)
  public String handleException(
      StudentNotFoundException ex) {

    return ex.getMessage();
  }
}
```

9. Explain testing in Spring Boot using JUnit with lifecycle methods.

1. Testing in Spring Boot is the process of checking whether the application works correctly as expected. It helps developers detect bugs and improve software quality.
2. Spring Boot provides built-in support for testing using JUnit and Spring testing tools. JUnit is mainly used for unit testing methods and application components.
3. In JUnit, test methods are marked using the @Test annotation. Assertion methods like assertEquals() are used to compare expected and actual outputs.
4. Example test method:

```
@Test
public void testAdd() {
  assertEquals(5, add(2,3));
}
```

5. JUnit lifecycle methods control what happens before and after test execution. These methods help initialize and clean resources during testing.
6. Lifecycle annotations include @BeforeAll, @BeforeEach, @AfterEach, and @AfterAll. They define the execution order of setup and cleanup methods.
7. Example lifecycle methods:

```
@BeforeAll
static void setup() {
  System.out.println("Before all tests");
}

@BeforeEach
void init() {
  System.out.println("Before each test");
}

@AfterEach
void cleanup() {
  System.out.println("After each test");
}

@AfterAll
static void finish() {
  System.out.println("After all tests");
}
```

8. The lifecycle order is @BeforeAll, @BeforeEach, @Test, @AfterEach, and @AfterAll. This structure helps maintain organized and reliable testing in Spring Boot applications.

10. Explain how to test REST controllers and importance of automated testing.

1. REST controller testing is used to verify whether API endpoints return correct responses. It ensures that APIs work properly before deployment.
2. In Spring Boot, REST controllers are tested using @SpringBootTest, @AutoConfigureMockMvc, and MockMvc. These tools help test APIs without running the actual server.
3. MockMvc is used to simulate HTTP requests and check API responses. It allows developers to test controller methods efficiently.
4. Example controller:

```
@GetMapping("/hello")
public String hello() {
  return "Hello";
}
```

5. Example test case:

```
@SpringBootTest
@AutoConfigureMockMvc
public class ControllerTest {

  @Autowired
  private MockMvc mockMvc;

  @Test
  public void testHello() throws Exception {

    mockMvc.perform(get("/hello"))
    .andExpect(status().isOk())
    .andExpect(content().string("Hello"));
  }
}
```

6. In this example, status().isOk() checks whether the response status is 200 OK. The content().string() method checks the returned response body.
7. Automated testing means running test cases automatically instead of checking functionality manually. It saves time and reduces human errors during testing.
8. Automated testing helps detect bugs early and ensures existing functionality still works after code changes. It improves software quality, reliability, and developer confidence in Spring Boot applications.